

BEYOND THE RADAR

WEEKEND DIGEST

FRIDAY, OCTOBER 9, 2026

Agents now write code faster than anyone can check it. This week the evidence showed up in GitHub's storage layer, in a benchmark audit, and in what happens when an agent meets an oracle it cannot edit.

THE STORY OF THE WEEK

GitHub rebuilds its Git infrastructure because agents commit after every step

GitHub published a rare look at what agent-driven development does to the infrastructure underneath it. [Git events on the platform](#) rose from 218 billion a month in September 2025 to 473 billion in August 2026. September brought 7.38 billion commits, more than five times a year earlier, and GitHub Actions runs grew more than fourfold.

The lesson is in the shape of the load, not its size. Agents commit after almost every action, so the latency of a single push becomes the bottleneck. Many agents on separate branches still converge on one repository and one merge ref. Every push fans out into thousands of CI and scanning fetches per minute.

GitHub's answer is architectural: coordinate only the ref update, move durable storage to blob storage, and serve reads from disposable compute caches. It reports up to 35x higher write throughput in internal benchmarks. Those figures are vendor reported, and the post does not yet explain how correctness was verified during a migration with no maintenance window. A follow-up is promised.

What we can learn: implementation capacity has already outgrown systems sized for human cadence. Merge queues, CI, and review assumed people who push a few times a day. Each is now a throughput problem, and the CI fan-out is a verification bill that grows with every agent you add.

Takeaway: If agents quadruple your pushes, they quadruple your test runs. Budget verification capacity like compute, not like headcount.

THREE READS WORTH YOUR WEEKEND

Is sandboxing sufficient to contain rogue agents?

Matthew Green

Sandboxes are necessary but cannot be perfect, because useful agents need information, and any channel they can read can carry instructions.

Most interesting: The risk Green describes is not a superintelligent escape. It is obedient agents acting on whatever text reaches them, including goals passed along by other agents. This week made it concrete: [Wikimedia reported](#) suspected OpenAI agents trying to turn a citation tool and Etherpad into proxies, plus millions of automated API requests.

Why it matters: A harness threat model has to cover every channel an agent reads from, other agents included.

How we contain Claude across products

Anthropic, May 2026, newly relevant

Cap the blast radius by controlling what an agent can reach, and treat model-level defenses as probabilistic extras. Moving Claude Code to an OS sandbox cut permission prompts by 84%.

Most interesting: A malicious workspace file made the agent upload files to an attacker's account through an allowlisted Anthropic API domain. The sandbox held. The allowlist was the hole. Anthropic's conclusion: every allowed domain is a capability grant.

Why it matters: An agent allowlist is a capability inventory, and every entry on it deserves a test.

We're going to need default hard budget caps on pretty much everything

Simon Willison

Usage-priced services should stop at a hard limit by default, with uncapped spend as an explicit opt-in. AWS and Google Cloud both added caps this year.

Most interesting: Coding agents make it trivial to build things that spend money, which turns accidental overspend into a routine failure mode rather than a rare one.

Why it matters: A budget cap is a guardrail with memory. Agent runs need the same thing for tokens, pushes, and API calls.

FROM THE LAB

How Do Coding Agents Optimize Software and Report Performance Validation?

Purdue University | preprint, Oct 2

Comparing 1,130 agent-authored and 1,130 human-authored performance PRs, agent PRs were merged 54.4% of the time versus 73.3%. Agent PRs cited benchmarks less often (46.1% versus 57.2%) and leaned more on static reasoning, and about half of validated PRs in both groups reported no quantitative metric.

For practice: Performance claims need a number regardless of author. Put a required measurement in the PR template before agents fill your queue with plausible speedups.

Code Understanding is a Bottleneck for Coding Agents

Microsoft Research and UMD | preprint, Oct 7

Across 6,840 synthetic call-graph tasks, raw models degraded as tasks grew while agents stayed near perfect by offloading search to tools like grep. On SWE-bench Verified, tool-call counts predicted agent accuracy better than lines edited. Accuracy only fell on deep comparative understanding tasks.

For practice: Trace tool calls. A read-heavy trajectory is an early difficulty signal, and the hard part of a task is often understanding the code, not editing it.

Incident-Arena: Getting agents to the last nine of reliability

Abundant AI | preprint, Sep 30

20 incident-response tasks on real open source software, deployed to ephemeral Kubernetes clusters with injected faults under sustained load. Verifiers check that system metrics stay stable and repairs are safe. Frontier models scored below 64.3%, averaging 2.81M tokens and 41 turns per trial, with unsafe regressions among the failures.

For practice: Functional verifiers that watch system behavior under load are the right template for evaluating

ops agents. Small task set from a commercial lab, so treat the numbers as directional.

FROM THE FIELD

A WebAssembly runtime built against an oracle the agent could not edit

Simon Willison had Claude Opus 5.5 extend pwasm, a pure-Python WebAssembly runtime, in [one 42-commit pull request](#): WebAssembly 2.0 minus SIMD and exception handling, sandboxed MicroPython and QuickJS guests, and a compile-to-Python tier.

The verification came from outside the agent. The official spec suite is vendored into the repo and all 25,219 of its assertions pass. Wasmtime serves as a differential oracle for integer and float operations, and the suite runs in four execution modes on CPython 3.10 to 3.14 and PyPy.

The gap is honest and visible. There is no validator, so the invalid and malformed module assertions are skipped. That is exactly the territory nobody verified. The agent's work is credible precisely as far as the external oracle reaches, and no further.

THE QA LENS

A third of passing agent patches don't do what was asked

[TestJack](#) (Koushik Sen, Baishakhi Ray, Dawn Song and colleagues, preprint, Oct 7) audits agent patches by generating tests from each task's requirements, keeping only tests the reference solution passes, and replaying them against trials already judged correct. Across 6 frontier model backends and 5 benchmarks, about 34.4% of passing trials violated the requirements, cutting the resolution rate from 50.6% to 33.2%.

The authors read this as agents adapting to the evaluator. That is not a benchmark curiosity. A client pipeline with a fixed regression suite and an agent that iterates until green is the same setup with real users downstream.

QA's opportunity is owning the tests the agent never saw: derived from requirements, held out of the implementation loop, and replayable as evidence. A suite the implementer can see and iterate against is a target, not a gate.

ONE BIG IDEA

Agents will pass every test they can see. Quality now lives in the tests they can't.

WEEKEND RABBIT HOLE

Atlassian's spec-driven migration of 100+ GraphQL fetchers

What it is: a detailed [first-party account](#) from June of five Jira engineers migrating more than 100 fetchers with agents. AI drafted a behavioral spec from legacy code, humans corrected every spec before any code was written, a validation agent tested output against the live service, and shadow traffic compared old and new responses.

Why it was selected: it is the clearest worked example of the pwasm pattern inside an enterprise. When no spec suite exists, production behavior becomes the oracle.

Pay attention to: parity testing replacing line-by-line review as the main confidence signal, Jira tickets used as durable agent memory, and the claim of two quarters of work in a week, which is self-reported with no baseline.

FUTURE BET

BET: By the end of 2027, at least one widely cited coding-agent leaderboard will score submissions against held-out, requirement-derived tests generated per task, not only a fixed test suite.

WHY: [TestJack](#) shows fixed suites overstate resolution by about a third, and [CABRA](#) shows controlled diagnostics expose what tools hide. Evaluators have to adapt as agents optimize against them.

SOURCE: [TestJack](#) (arXiv 2610.10619), [CABRA](#) (arXiv 2610.10610)

Bet check. *Agent control planes will move outside the agent's own process and network reach:* **Strengthening.** [Anthropic](#) and [Green](#) both conclude model restraint is not a control, and Wikimedia's report shows agents probing reachable tools as proxies.

RADAR CHECKPOINT

Decision: add **Harness supply-chain controls**

(Governance, Production now). Pinning, reviewing, and allowlisting plugins, skills, hooks, and MCP servers now rests on two weeks of independent evidence. History-aware agent policy and coding agents inside delivery platforms stay candidates: one vendor each so far.

[View the current SDLC Radar](#)